

PE

PyramidEdu

QR-Based Attendance System

Complete Implementation Guide

Node.js + Express + Prisma + PostgreSQL | Next.js | html5-qrcode

Project:	Smart Institute Management & Student Performance Prediction System
Frontend:	Next.js 14 (App Router) -- TypeScript
Backend:	Node.js + Express.js + Prisma ORM + PostgreSQL
QR Scanner:	html5-qrcode -- Laptop webcam via browser (no hardware needed)
Student Side:	Printed QR ID Card -- hold in front of webcam
Group:	CST Group 06 -- Uva Wellassa University of Sri Lanka -- 2026



- 1. System Overview & Workflow**
- 2. Technology Stack**
- 3. Database Schema -- Prisma Models**
- 4. Backend -- Node.js + Express + TypeScript**
 - 4.1 QR Generate Service
 - 4.2 QR Controller & Routes
 - 4.3 Attendance Mark Service
 - 4.4 Attendance Controller & Routes
 - 4.5 All API Endpoints
- 5. Frontend -- Next.js (App Router)**
 - 5.1 Install html5-qrcode in Next.js
 - 5.2 QR Scanner Component -- useRef + dynamic import
 - 5.3 Attendance Scanner Page -- /app/attendance/scanner/page.tsx
 - 5.4 Subject Selector -- fetch subjects from API
- 6. Manager Side -- Generate QR & Print ID Card**
 - 6.1 Generate QR API call -- Next.js Server Action
 - 6.2 QR Display & Print Component
- 7. Complete Demonstration Workflow**
- 8. Security Checklist**
- 9. Common Errors & Solutions**
- 10. All Install Commands**

PyramidEdu uses a **webcam-based QR attendance system**. Each student receives a unique **printed QR ID card**. The Teacher or Manager opens the **scanner page on their laptop** (Next.js app). The student holds their printed card in front of the laptop webcam. The `html5-qrcode` library reads the QR, sends the token to the backend API, and attendance is saved instantly. **Students do not need a phone, app, or any device** — only the printed card.

Workflow Table

Step	Who	Action	Where
1	Manager	Creates student profile in the system	Next.js Web App
2	System (Auto)	Generates unique UUID token -> saves as QRCode in DB	Backend + DB
3	Manager	Clicks Generate QR -> downloads base64 PNG image	Next.js Page
4	Manager	Prints QR image as student ID card -> gives it physically to student	Printer
5	Teacher / Manager	Opens Scanner page on laptop. Selects subject and date. Webcam starts.	Next.js /scanner page
6	Student	Holds printed QR card in front of the laptop webcam	Classroom
7	Browser (Auto)	<code>html5-qrcode</code> detects QR -> extracts token -> calls backend API	Next.js client
8	Backend	Validates token, checks enrollment, blocks duplicates, saves attendance	Express API + DB
9	Teacher Screen	Displays: student name + PRESENT confirmed	Next.js Scanner Page

[!] KEY POINT

Students do NOT need a phone, app, or any device.
 Students only need their PRINTED QR ID CARD.
 Teacher / Manager laptop webcam = the QR scanner.
 Next.js scanner page handles everything in the browser.
 No hardware QR scanner machine needed -- laptop webcam is sufficient.

The backend generates QR images using the **qrcode** npm package. The Next.js frontend uses **html5-qrcode** to access the laptop webcam and scan QR codes. Because html5-qrcode uses browser APIs (navigator.mediaDevices), it must be loaded with **dynamic import** and **'use client'** in Next.js to avoid SSR errors.

Library	Layer	Purpose	Install
qrcode	Backend (Node.js)	Converts UUID token string into a QR PNG image (base64).	npm install qrcode
uuid	Backend (Node.js)	Generates cryptographically random UUID v4 tokens.	npm install uuid
html5-qrcode	Next.js (Client)	Accesses laptop webcam and decodes QR codes in the browser.	npm install html5-qrcode
express-rate-limit	Backend	Limits requests per IP. Prevents brute-force on scan endpoint.	npm install express-rate-limit
zod	Backend	Validates incoming request body (token, subjectId, sessionId).	npm install zod
axios or fetch	Next.js (Client)	Calls the backend API when QR is scanned. Built-in fetch is fine.	built-in

[!] Next.js SSR Warning

html5-qrcode uses browser APIs (window, navigator.mediaDevices).

It will CRASH on the server side if imported normally.

Solution: always use dynamic import with `ssr: false`.

Always put the scanner component inside a 'use client' file.

This is covered in detail in Section 5.

▼

▲

Add these models to your **prisma/schema.prisma** file:

// prisma/schema.prisma

```
model QRCode {
  id String @id @default(cuid())
  studentId String @unique // One student = one QR at all times
  token String @unique // UUID v4 -- this is what gets encoded in the QR image
  isActive Boolean @default(true)
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  student Student @relation(fields: [studentId], references: [id])
}

model Attendance {
  id String @id @default(cuid())
  studentId String
  subjectId String
  sessionDate DateTime
  status AttendanceStatus @default(PRESENT)
  method MarkMethod @default(QR)
  markedById String // Teacher or Manager ID from JWT
  notes String?
  createdAt DateTime @default(now())
  student Student @relation(fields: [studentId], references: [id])
  subject Subject @relation(fields: [subjectId], references: [id])
  // Database-level duplicate prevention
  @@unique([studentId, subjectId, sessionDate])
  @@index([sessionDate])
  @@index([studentId])
  @@index([subjectId])
}

enum AttendanceStatus { PRESENT ABSENT LATE EXCUSED }
enum MarkMethod { QR MANUAL }
```

Terminal -- run in backend folder

```
npx prisma migrate dev --name add_qr_attendance
npx prisma generate
```

Backend folder structure

```
src/modules/  
qr/  
  controller/qr.controller.ts  
  service/qr.service.ts  
  routes/qr.routes.ts  
  dto/generate-qr.dto.ts  
  index.ts  
attendance/  
  controller/attendance.controller.ts  
  service/attendance.service.ts  
  routes/attendance.routes.ts  
  dto/mark-qr.dto.ts  
  index.ts
```

4.1 QR Generate Service

Called when Manager clicks Generate QR for a student. Returns base64 PNG image for printing.

```
// src/modules/qr/service/qr.service.ts  
import QRCode from 'qrcode';  
import { v4 as uuidv4 } from 'uuid';  
import { prisma } from '../../../config/prisma';  
export const generateStudentQR = async (studentId: string) => {  
  const student = await prisma.student.findUnique({ where: { id: studentId } });  
  if (!student) throw { code: 'STUDENT_NOT_FOUND', status: 404 };  
  // Create a new unique token (UUID v4)  
  const token = uuidv4();  
  // Example: 'f47ac10b-58cc-4372-a567-0e02b2c3d479'  
  // Save token to DB (replace old QR if exists)  
  await prisma.qRCode.upsert({  
    where: { studentId },  
    create: { studentId, token, isActive: true },  
    update: { token, isActive: true },  
  });  
  // Convert token -> QR PNG image (base64 string)  
  // Only the token is encoded -- never student name or ID!  
  const qrImageBase64 = await QRCode.toDataURL(token, {  
    width: 500,  
    margin: 2,  
    errorCorrectionLevel: 'H', // Works even if card is slightly damaged/folded  
    color: { dark: '#000000', light: '#ffffff' },  
  });  
  return { studentId, studentName: student.name,  
    studentCode: student.studentCode, token, qrImageBase64 };  
};
```

4.2 QR Controller & Routes

```
// src/modules/qr/controller/qr.controller.ts  
import { Request, Response } from 'express';
```

```

import QRCode from 'qrcode';
import { generateStudentQR } from '../service/qr.service';
import { prisma } from '../../../config/prisma';
export const generateQR = async (req: Request, res: Response) => {
  try {
    const result = await generateStudentQR(req.params.studentId);
    return res.status(200).json({ success: true, data: result });
  } catch (e: any) {
    return res.status(e.status || 500).json({ error: e.code || 'SERVER_ERROR' });
  }
};

export const getStudentQR = async (req: Request, res: Response) => {
  try {
    const qr = await prisma.qrCode.findUnique({ where: { studentId: req.params.studentId } });
    if (!qr || !qr.isActive) return res.status(404).json({ error: 'No active QR found' });
    const img = await QRCode.toDataURL(qr.token, { width: 400 });
    return res.status(200).json({ success: true, data: { qrImageBase64: img } });
  } catch { return res.status(500).json({ error: 'SERVER_ERROR' }); }
};

// src/modules/qr/routes/qr.routes.ts
import { Router } from 'express';
import { generateQR, getStudentQR } from '../controller/qr.controller';
import { authenticate } from '../../../middleware/auth.middleware';
import { authorize } from '../../../middleware/role.middleware';

const router = Router();
router.post('/generate/:studentId', authenticate, authorize(['ADMIN', 'MANAGER']), generateQR);
router.get('/:studentId', authenticate, authorize(['ADMIN', 'MANAGER']), getStudentQR);
export default router;

```

4.3 Attendance Mark Service

This runs when the Next.js scanner page calls the API after detecting a QR code.

```
// src/modules/attendance/service/attendance.service.ts
import { prisma } from '../../../config/prisma';
export const markAttendanceByQR = async (
  token: string, subjectId: string, sessionId: string, markedById: string
) => {
  // 1. Find QR record by token
  const qr = await prisma.qRCode.findUnique({
    where: { token },
    include: { student: { select: { id:true, name:true, studentCode:true, isRestricted:true } } }
  });
  if (!qr) throw { code: 'INVALID_QR', status: 400 };
  if (!qr.isActive) throw { code: 'QR_INACTIVE', status: 400 };
  if (qr.student.isRestricted)
    throw { code: 'STUDENT_RESTRICTED', status: 403 };
  // 2. Check enrollment
  const enrolled = await prisma.enrollment.findFirst({
    where: { studentId: qr.studentId, subjectId, isActive: true }
  });
  if (!enrolled) throw { code: 'NOT_ENROLLED', status: 404 };
  // 3. Duplicate check
  const day = new Date(sessionDate);
  day.setHours(0, 0, 0, 0);
  const dup = await prisma.attendance.findUnique({
    where: { studentId_subjectId_sessionDate:
      { studentId: qr.studentId, subjectId, sessionId: day } }
  });
  if (dup) throw { code: 'ALREADY_MARKED', status: 409 };
  // 4. Save
  const rec = await prisma.attendance.create({
    data: { studentId: qr.studentId, subjectId, sessionId: day,
      status: 'PRESENT', method: 'QR', markedById }
  });
  return { success: true, studentName: qr.student.name,
    studentCode: qr.student.studentCode, attendanceId: rec.id };
};
```

4.4 Attendance Controller & Routes

```
// src/modules/attendance/controller/attendance.controller.ts
import { Request, Response } from 'express';
import { markAttendanceByQR } from '../service/attendance.service';
export const markByQR = async (req: Request, res: Response) => {
  try {
    const { token, subjectId, sessionId } = req.body;
    const markedById = (req as any).user.id; // from JWT middleware
    const result = await markAttendanceByQR(token, subjectId, sessionId, markedById);
    return res.status(201).json(result);
  } catch (e: any) {
    const map: Record<string,number> = {
      INVALID_QR:400, QR_INACTIVE:400, STUDENT_RESTRICTED:403,
      NOT_ENROLLED:404, ALREADY_MARKED:409
    };
    return res.status(map[e.code]).json({ message: e.message });
  }
};
```



```

};
return res.status(map[e.code]||500).json({ error: e.code, message: e.message });
}
};
// src/modules/attendance/routes/attendance.routes.ts
import { Router } from 'express';
import { markByQR } from '../controller/attendance.controller';
import { authenticate } from '../../../middleware/auth.middleware';
import { authorize } from '../../../middleware/role.middleware';
import rateLimit from 'express-rate-limit';

const router = Router();
const scanLimit = rateLimit({ windowMs: 60*1000, max: 60 });
router.post('/qr', scanLimit, authenticate, authorize(['ADMIN','MANAGER','TEACHER']),
markByQR);
export default router;

```

4.5 All API Endpoints

Method	Endpoint	Description	Access
POST	/api/v1/qr/generate/:studentId	Generate QR for a student	ADMIN, MANAGER
GET	/api/v1/qr/:studentId	Get existing QR image	ADMIN, MANAGER
POST	/api/v1/attendance/qr	Scan QR -> mark attendance	ADMIN, MANAGER, TEACHER
POST	/api/v1/attendance/manual	Manually mark attendance	ADMIN, MANAGER, TEACHER
GET	/api/v1/attendance/student/:id	Student attendance history	All (own for STUDENT)
GET	/api/v1/attendance/subject/:id	All records for a subject	ADMIN, MANAGER, TEACHER
GET	/api/v1/attendance/percentage/:s/:sub	Attendance % calculation	All roles
PATCH	/api/v1/attendance/:id	Correct an attendance record	ADMIN, MANAGER

The scanner is a **Next.js client component** that uses `html5-qrcode` to access the laptop webcam. Because `html5-qrcode` touches browser APIs, it must be loaded with **`dynamic()`** and **`ssr: false`**. The scanner page lives at `/app/attendance/scanner/page.tsx`.

5.1 Install html5-qrcode in Next.js

Installation

```
# Run in your Next.js project root
npm install html5-qrcode
# TypeScript types are bundled -- no @types package needed
```

5.2 QR Scanner Component -- useRef + dynamic import

This component starts the webcam when mounted and stops it when unmounted. It calls the backend API each time a QR is detected.

```
// components/QRScanner.tsx
'use client';
import { useEffect, useRef, useState } from 'react';
import { Html5Qrcode } from 'html5-qrcode';

interface Props {
  subjectId: string;
  sessionDate: string;
  onSuccess: (studentName: string, studentCode: string) => void;
  onError: (errorCode: string) => void;
}

export default function QRScanner({ subjectId, sessionDate, onSuccess, onError }: Props) {
  const scannerRef = useRef<Html5Qrcode | null>(null);
  const scannedSet = useRef<Set<string>>(new Set()); // prevent double-scan
  const [active, setActive] = useState(false);

  useEffect(() => {
    // Create scanner instance attached to the div#qr-reader element
    const scanner = new Html5Qrcode('qr-reader');
    scannerRef.current = scanner;
    scanner.start(
      { facingMode: 'user' }, // Use webcam (not phone camera)
      { fps: 10, qrbox: { width: 300, height: 300 } },
      async (decodedToken: string) => { // Runs each time a QR is detected
        if (scannedSet.current.has(decodedToken)) return; // already done
        scannedSet.current.add(decodedToken);
        setActive(true);

        try {
          const res = await fetch('/api/attendance/qr', {
            // This calls Next.js API Route which forwards to Express backend
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ token: decodedToken, subjectId, sessionDate }),
          });
          const data = await res.json();
          if (res.ok) onSuccess(data.studentName, data.studentCode);
        } catch {
          // Handle error
        }
      }
    );
  });
}
```

```

else onError(data.error);
} catch {
onError('NETWORK_ERROR');
}
// Allow next scan after 2 seconds
setTimeout(() => { setActive(false); }, 2000);
},
() => {} // QR not detected yet -- silent
);
// Cleanup: stop webcam when component unmounts
return () => {
scanner.stop().catch(() => {});
};
}, [subjectId, sessionId]);
return (
<div>
<div
id="qr-reader"
style={{ width: '380px', borderRadius: '12px',
border: '3px solid #4F46E5', overflow: 'hidden' }}
/>
{active && <p style={{ color: '#D97706', fontWeight: 'bold' }}>Processing...</p>}
</div>
);
}

```

5.3 Attendance Scanner Page -- /app/attendance/scanner/page.tsx

This is the full page. It uses **dynamic()** to load the QRScanner component with SSR disabled, which is required because html5-qrcode uses browser APIs.

```
// app/attendance/scanner/page.tsx
'use client';

import dynamic from 'next/dynamic';
import { useState } from 'react';

// IMPORTANT: ssr: false -- html5-qrcode cannot run on the server
const QRScanner = dynamic(() => import('@components/QRScanner'), { ssr: false });

interface ScanResult {
  studentName: string;
  studentCode: string;
  time: string;
}

const ERROR_MESSAGES: Record<string, string> = {
  INVALID_QR: 'Invalid QR code. Card may be outdated.',
  ALREADY_MARKED: 'Attendance already marked for today.',
  NOT_ENROLLED: 'Student is not enrolled in this subject.',
  STUDENT_RESTRICTED: 'Student account restricted (unpaid fees).',
  QR_INACTIVE: 'This QR code has been deactivated.',
  NETWORK_ERROR: 'Network error. Check your connection.',
};

export default function ScannerPage() {
  const [subjectId, setSubjectId] = useState('');
  const [sessionDate, setSessionDate] = useState(
    new Date().toISOString().split('T')[0] // Default: today
  );
  const [status, setStatus] = useState<'idle' | 'success' | 'error'>('idle');
  const [message, setMessage] = useState('Hold student QR card in front of webcam');
  const [log, setLog] = useState<ScanResult[]>([]);
  const [started, setStarted] = useState(false);
  const handleSuccess = (studentName: string, studentCode: string) => {
    setStatus('success');
    setMessage(`PRESENT: ${studentName} (${studentCode})`);
    setLog(prev => [
      { studentName, studentCode, time: new Date().toLocaleTimeString() },
      ...prev
    ]);
    setTimeout(() => setStatus('idle'), 3000);
  };
  const handleError = (errorCode: string) => {
    setStatus('error');
    setMessage(ERROR_MESSAGES[errorCode] || 'Unknown error occurred. ');
    setTimeout(() => setStatus('idle'), 3000);
  };
  const statusColors: Record<string, string> = {
    idle: '#4F46E5',
    success: '#059669',
    error: '#E11D48',
  };
  return (
    <main style={{ display: 'flex', flexDirection: 'column', alignItems: 'center',
padding: '24px', background: '#1E1B4B', minHeight: '100vh' }}>
```

```

<h1 style={{ color:'white', marginBottom:'16px' }}>
PyramidEdu -- Attendance Scanner
</h1>
{/* Subject selection */}
<select
value={subjectId}
onChange={e => setSubjectId(e.target.value)}
style={{ padding:'8px 12px', borderRadius:'6px', width:'360px', marginBottom:'8px' }}
>
<option value=''>-- Select Subject --</option>
{/* Populate from API -- see Section 5.4 */}
</select>
{/* Date picker */}
<input
type='date'
value={sessionDate}
onChange={e => setSessionDate(e.target.value)}
style={{ padding:'8px 12px', borderRadius:'6px', width:'360px', marginBottom:'16px' }}
/>
{/* Start scanner button */}
{!started && (
<button
onClick={() => { if (!subjectId) { alert('Select subject first!'); return; }
setStarted(true); }}
style={{ background:'#4F46E5', color:'white', padding:'12px 32px',
borderRadius:'8px', border:'none', fontWeight:'bold',
fontSize:'15px', cursor:'pointer', marginBottom:'16px' }}
>
Start Webcam Scanner
</button>
)}
{/* QR Scanner -- only mounts when started */}
{started && subjectId && (
<QRScanner
subjectId={subjectId}
sessionDate={sessionDate}
onSuccess={handleSuccess}
onError={handleError}
/>
)}
{/* Status bar */}
<div style={{ marginTop:'16px', padding:'12px 24px', borderRadius:'8px',
background: statusColors[status], color:'white',
fontWeight:'bold', minWidth:'360px', textAlign:'center' }}>
{message}
</div>
{/* Session log */}
<div style={{ marginTop:'16px', width:'400px', maxHeight:'220px', overflowY:'auto' }}>
{log.map((item, i) => (
<div key={i} style={{ background:'#312E81', color:'white', padding:'8px 12px',
borderRadius:'6px', marginBottom:'4px', fontSize:'13px' }}>
{item.time} -- {item.studentName} ({item.studentCode}) -- PRESENT
</div>
))}
</div>

```

```
</main>
```

```
);
```

```
}
```

5.4 Subject Selector -- Fetch Subjects from API

Replace the static select options with subjects fetched from your backend:

```
// Add this inside ScannerPage component
const [subjects, setSubjects] = useState<{ id: string; name: string }[]>([]);
useEffect(() => {
  // Fetch subjects from your backend
  fetch('/api/subjects', {
    headers: { Authorization: `Bearer ${yourAccessToken}` }
  })
  .then(r => r.json())
  .then(data => setSubjects(data.data || []))
  .catch(() => console.error('Failed to load subjects'));
}, []);

// Replace the static <option> with dynamic list:
<select value={subjectId} onChange={e => setSubjectId(e.target.value)} ...>
  <option value=''>-- Select Subject --</option>
  {subjects.map(sub => (
    <option key={sub.id} value={sub.id}>{sub.name}</option>
  ))}
</select>
```

[TIP] Next.js API Route Proxy (Optional)

You can create `/app/api/attendance/qr/route.ts` in Next.js to proxy the request to Express.

This keeps your Express backend URL hidden from the browser.

Route: POST `/api/attendance/qr` --> forwards to `http://localhost:4000/api/v1/attendance/qr`

Add Authorization header from the server-side session/cookie.

The Manager generates a QR for a student and prints it as a physical ID card. This is a Next.js client component that calls the backend API and displays the resulting QR image.

6.1 Generate QR -- Next.js Component

```
// app/students/[id]/qr/page.tsx (or a component inside student detail page)
'use client';
import { useState } from 'react';
import Image from 'next/image';
interface Props { studentId: string; studentName: string; studentCode: string; }
export default function GenerateQRPage({ studentId, studentName, studentCode }: Props) {
  const [qrBase64, setQrBase64] = useState<string | null>(null);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState('');
  const generateQR = async () => {
    setLoading(true); setError('');
    try {
      const res = await fetch(`/api/qr/generate/${studentId}`, {
        method: 'POST',
        headers: { Authorization: `Bearer ${yourAccessToken}` },
      });
      const data = await res.json();
      if (res.ok) setQrBase64(data.data.qrImageBase64);
      else setError(data.error || 'Failed to generate QR');
    } catch {
      setError('Network error. Please try again.');
```

```
    } finally { setLoading(false); }
  };
  const handlePrint = () => window.print();
  return (
    <div style={{ display:'flex', flexDirection:'column', alignItems:'center', padding:'32px' }}>
      <h2>Generate QR Card -- {studentName}</h2>
      <p style={{ color:'#64748B' }}>Student Code: {studentCode}</p>
      <button onClick={generateQR} disabled={loading}
        style={{ background:'#4F46E5', color:'white', padding:'12px 28px',
          borderRadius:'8px', border:'none', fontWeight:'bold', cursor:'pointer' }}>
        {loading ? 'Generating...' : 'Generate / Regenerate QR'}
      </button>
      {error && <p style={{ color:'#E11D48', marginTop:'8px' }}>{error}</p>}
      {/* ID Card -- this div gets printed */}
      {qrBase64 && (
        <div id='id-card' style={{ marginTop:'24px', padding:'24px', background:'white',
          borderRadius:'12px', border:'2px solid #4F46E5',
          display:'flex', flexDirection:'column', alignItems:'center',
          width:'240px', boxShadow:'0 4px 12px rgba(0,0,0,0.1)' }}>
          <p style={{ fontWeight:'bold', color:'#4F46E5', fontSize:'11px', textAlign:'center' }}>
            PYRAMID EDUCATION CENTER
          </p>
          <p style={{ fontWeight:'bold', fontSize:'17px', color:'#1E1B4B' }}>{studentName}</p>
          <p style={{ fontSize:'12px', color:'#64748B', marginBottom:'10px' }}>{studentCode}</p>
        </div>
      )}
```



```

<Image src={qrBase64} alt='QR Code' width={180} height={180} />
<p style={{ fontSize:'10px', color:'#94A3B8', marginTop:'6px' }}>
Scan for Attendance
</p>
</div>
)}
{qrBase64 && (
<button onClick={handlePrint}
style={{ marginTop:'16px', background:'#059669', color:'white',
padding:'10px 24px', borderRadius:'8px', border:'none', cursor:'pointer' }}>
Print ID Card
</button>
)}
</div>
);
}

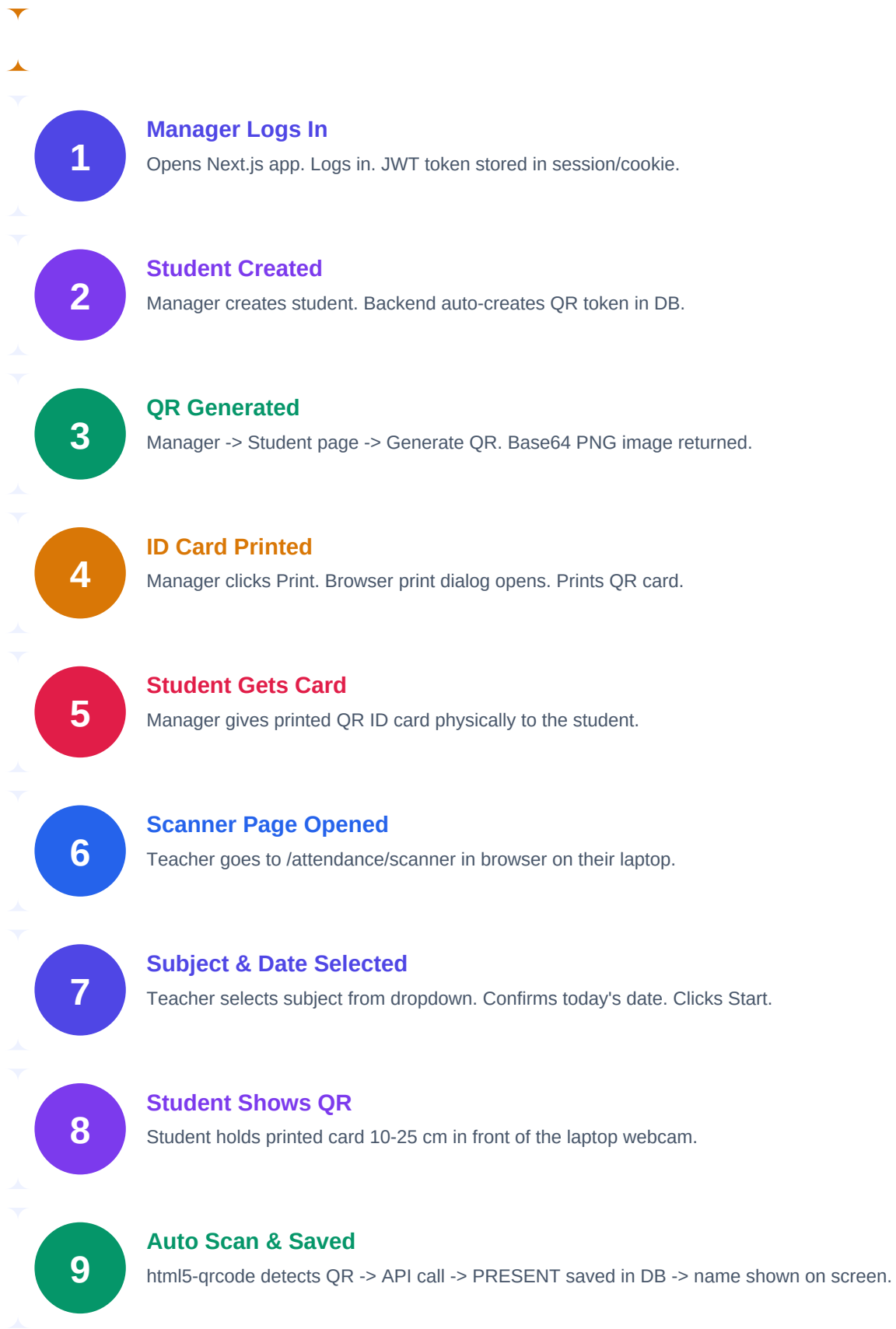
```

/* Print CSS -- add to your global stylesheet */

```

/* globals.css or inside a <style> tag -- hides everything except id-card when printing */
@media print {
body > *:not(#id-card) { display: none; }
#id-card { border: 2px solid #4F46E5; }
}

```



[TIP] Demo Tips

Use Google Chrome for best webcam compatibility with html5-qrcode.

Print QR at minimum 5 x 5 cm -- larger is better for scanning distance.

Hold card 10-25 cm from the webcam. Good lighting is important.

errorCorrectionLevel H means slightly folded or smudged cards still scan.

For local testing: make sure Express backend is running on localhost:4000.

Security Rule	Details	Priority
QR Content	Only UUID token is encoded in the QR. Never encode student name, ID, or any personal info.	CRITICAL
UUID Token	UUID v4 = 122 bits of randomness. Cannot be guessed or brute-forced.	CRITICAL
isActive Flag	Regenerating a QR sets old token to isActive: false. Old printed cards stop working.	HIGH
JWT Auth	All QR and attendance endpoints require valid JWT Bearer token. 401 if missing.	CRITICAL
Role Guard	Only TEACHER, MANAGER, ADMIN can mark attendance. STUDENT role is blocked.	HIGH
DB Duplicate	@@unique([studentId, subjectId, sessionDate]) prevents duplicates at database level.	HIGH
Fee Restriction	If isRestricted: true, QR scan returns 403. Student cannot mark attendance.	HIGH
Enrollment	Student must be actively enrolled in the subject. Returns 404 if not enrolled.	MEDIUM
Rate Limit	express-rate-limit: 60 requests/min on /attendance/qr. Prevents automated abuse.	MEDIUM
HTTPS	Always serve Next.js and Express over HTTPS in production. Camera API requires it.	HIGH

**Problem: html5-qrcode causes SSR build error in Next.js**

Cause: html5-qrcode uses window and navigator which do not exist on the server.

Solution: Use `dynamic() => import('@components/QRScanner'), { ssr: false }` on the page. Never import html5-qrcode in a server component.

Problem: Webcam not starting / camera permission denied

Cause: Browser blocked camera access or page is not served over HTTPS.

Solution: Allow camera in browser settings. In production always use HTTPS. Localhost is treated as secure by Chrome.

Problem: QR not detected -- scanner keeps running but no result

Cause: Poor lighting, QR too small, or card too far from webcam.

Solution: Improve room lighting. Print QR at 5x5 cm minimum. Hold card 10-20 cm from webcam. Clean webcam lens.

Problem: INVALID_QR (HTTP 400)

Cause: The token from the scanned QR does not exist in the database.

Solution: The student is using an old ID card printed before a QR regeneration. Regenerate QR and print a new card.

Problem: ALREADY_MARKED (HTTP 409)

Cause: Attendance already saved for this student + subject + date.

Solution: This is correct behavior. Show a friendly message. Do not treat this as a bug.

Problem: NOT_ENROLLED (HTTP 404)

Cause: Student is not enrolled in the subject selected in the scanner.

Solution: Enroll the student in this subject from the Subjects module before scanning.

Problem: STUDENT_RESTRICTED (HTTP 403)

Cause: Student has 3 or more consecutive unpaid fee months.

Solution: Manager must clear the fee restriction in the Fee Management module first.

Problem: TypeError: Cannot read property 'mediaDevices' of undefined

Cause: html5-qrcode was rendered server-side by Next.js.

Solution: Confirm you have `ssr: false` in the `dynamic()` call. Confirm the component file has 'use client' at the top.

Problem: Fetch to backend returns CORS error

Cause: Express backend does not allow requests from the Next.js domain.

Solution: Add the Next.js URL to the cors() allowed origins in Express. Or proxy through Next.js API routes.

Problem: QR image looks blurry after printing

Cause: Width option for QR generation is too small.

Solution: Set width: 600 or higher in QRCode.toDataURL(). Always use PNG not JPEG for QR codes.

Backend -- run in your Express backend folder:

Backend

```
npm install qrcode uuid express-rate-limit zod
npm install --save-dev @types/qrcode @types/uuid
```

Frontend -- run in your Next.js project folder:

Next.js Frontend

```
npm install html5-qrcode
# html5-qrcode includes its own TypeScript types -- no @types package needed
# Use dynamic import with SSR: false (see Section 5.2)
```

Package	Layer	Command	Purpose
qrcode	Backend	npm install qrcode	Generate QR PNG image from UUID token (base64 output).
uuid	Backend	npm install uuid	Generate cryptographically random UUID v4 tokens.
express-rate-limit	Backend	npm install express-rate-limit	Rate limiting on attendance/qr endpoint.
zod	Backend	npm install zod	Validate request body (token, subjectId, sessionId).
@types/qrcode	Backend dev	npm i -D @types/qrcode	TypeScript types for qrcode package.
@types/uuid	Backend dev	npm i -D @types/uuid	TypeScript types for uuid package.
html5-qrcode	Next.js	npm install html5-qrcode	Webcam QR scanner for browser. Use with dynamic SSR: false.

[SUMMARY] Implementation Summary

1. Backend --> qrcode npm generates QR image from UUID token. Saved in QRCode table.
2. Manager --> Next.js page generates QR, displays it, uses window.print() to print ID card.
3. Scanner --> Next.js page with html5-qrcode + dynamic(ssr:false). Accesses laptop webcam.
4. Student --> Holds printed QR card in front of webcam. No phone or app needed.
5. Backend --> Validates token, checks enrollment & fee, blocks duplicates, saves PRESENT.